

DATA REPRESENTATION IN A COMPUTER.

Computers handle data by electrical components, e.g. Transistors, Semiconductors, Integrated Circuits or wires, which exist in two conditions (states), ON & OFF or “1” & “0”.

Inside the computer, data is represented by storage cells, which are either electronically charged or discharged.

Examples;

- ♦ In RAM, the cells can be charged and discharged at will, and this can be used to store different data items. The charged state of the cell can be represented by 1 (or ON), while the uncharged state by 0 (or OFF).
In ROM, the cells are permanently set to one state.
- ♦ A Transistor may be conducting or non-conducting.
- ♦ A Magnetic material may be magnetized in one direction or the other.
- ♦ A wire may or may not be carrying a current.

The Main memory of the computer can be considered as a collection of thousands of storage cells, each capable of representing a binary digit.

DATA TYPES.

Data is a term used to describe a set of facts. A single fact is known as **Datum**.

Data can be in 3 types or forms.

1. Numeric data (numbers)
2. Alphabetic data.
3. Alphanumeric data.

Numbers:

Numbers can be expressed as either;

- ❖ **Integers** - whole numbers, e.g., 124, -26, or
- ❖ **Real numbers** - numbers with decimal points, e.g., 1.23, -2.6.

Note. A whole number is Real if it is written with a decimal point, e.g., 25 is an integer, but 25.0 is real.

Alphabetic data:

This is data made from combination of alphabetic characters, such as names, title, marital status, e.g. “John”, “Prof.”, “Married”.

Alphanumeric data (Strings):

A **string** is any sequence of characters.

This is data made from combination of alphabetic characters, numerals and/or special characters.

Examples;

Names	- NG'ANG'A, ANN NDUVA
Address	- P.O BOX 299, UGUNJA
Date	- November 14, 1990
Account numbers	- AO133
Department	- Sales department
Messages	- Incorrect. Try again

Exercise.

1. Explain the terms Numeric and Alphanumeric. Illustrate your answers with appropriate examples.

THE BYTE.

The capacity of a computer memory can be measured in terms of **Bits** (individual memory cells), **Bytes** (groups of cells or bits) or **Words** (arrangement of the bytes).

- ❖ A **Bit (Binary digit)** is the number 0 or 1 in the representation of a value in binary notation.
- ❖ A **Byte** is a fixed number of adjacent bits that are operated on as a unit.
Usually, a byte is a group of 8 adjacent bits and can store one character, i.e. 1-byte stores 1 character.

The **Byte** is the most commonly used unit of measuring the capacity of a computer memory.

- ❖ A **Word** is a group of bits that the computer recognizes and executes (processes) at a time.

To represent characters, the bits are combined together. The group of bits representing characters can also be described as **Location**.

In a Character machine, a location has 6 bits that represents a Byte, while the Byte machines has 8 bits making up a byte.

Character machines

6 bits = 1 byte
12 bits = 2 bytes ($\frac{1}{2}$ Word)
24 bits = 4 bytes (1 Word)

Byte machines.

8 bits = 1 byte
16 bits = 2 bytes ($\frac{1}{2}$ Word)
32 bits = 4 bytes (1 Word)

The memory capacity can be expressed as 32k, 64k, 256k, etc. The '**K**' is a constant used to represent the Kilobyte, which is made up of 1,024 bytes.

Another unit that can be used to measure the memory is **Megabyte (MB)**. MB is used to denote (stand for / represent / indicate) a million bytes, i.e. 1024 K is equivalent to 1MB.

Note. Half of a byte is described as a **NIBBLE**. A Nibble can be made up of 3 bits, for Character machines and 4 bits for the Byte machines.

Exercise.

1. Data in a computer is represented in one major form. Define the term "*Data representation*" in a computer system.
2. Define the following terms:
 - (i). Bit.
 - (ii). Byte.
 - (iii). Character.
 - (iv). Word.
3. Explain the term "NIBBLES" as used in data representation in computers.

CODING OF DATA.

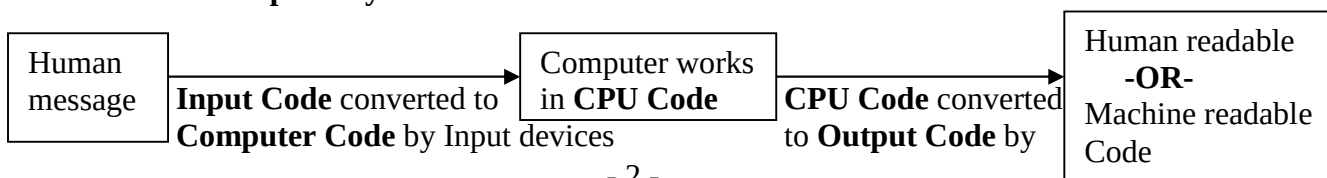
A computer can understand only one language consisting of two symbols, 0 & 1(Binary digits).

Since the computer cannot understand data represented in human languages (i.e. numerals 0 - 9, alphabets A – Z, and special symbols such as +, -, /, *, etc), it became necessary to change the data to binary form, a process known as **Coding of data**.

In other words, to make communication possible between computers and human beings, data must be coded in the form that can be understood by the computer and the information supplied by a computer (after processing) must be coded in the form that can be understood by the user.

The coding and decoding of data in a computer is done by the **Input/Output devices**.

Codes used in Computer systems.



Output devices

Data representation – The representation of normal data in some type of coded form, such as BCD, ASCII or EBCDIC.

NUMBER SYSTEMS.

The design and organization of a computer depends on the number system. The 4 number systems are:-

1. Binary number system (Base 2).
2. Octal number system (Base 8).
3. Decimal number system (Base 10).
4. Hexadecimal number system (Base 16).

Binary numbers are numbers to base 2, and use only two digits; 0 & 1.

Octal numbers, are numbers to base 8, and consists 8 digits (0 to 7).

The decimal number system consists of 10 digits, 0 to 9.

Hexadecimal numbers are numbers to base 16 and there must be 16 digits. The sixteen symbols used in the Hexadecimal system are; digits 0 to 9 & alphabets A to F.

Number System	Base	Digits and/or symbols represented
Binary	2	0, 1
Octal	8	0, 1, 2, 3, 4, 5, 6, 7
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Hexadecimal	16	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

DECIMAL NUMBER SYSTEM.

The decimal number system consists of 10 digits, 0 to 9.

In decimal system, each digit has;

- A **Digit value** (0 to 9).
- A **Positional value**, which is determined by how many places to the left or the right of the decimal point the digit is written.

Note. The Digit value & Positional value for each number system depend on the base of the number system.

Powers of the base increase as we move to the left and decrease as we move to the right.

Summary for decimal number system (Integers only).

7 th	6 th	5 th	4 th	3 rd	2 nd	1 st	Position
10^6	10^5	10^4	10^3	10^2	10^1	10^0	Power base
1000000	100000	10000	1000	100	10	1	Value

Summary for Integers as well as Real numbers.

4 th	3 rd	2 nd	1 st	▪	1 st	2 nd	3 rd	4 th	5 th	Position
10^3	10^2	10^1	10^0	▪	10^{-1}	10^{-2}	10^{-3}	10^{-4}	10^{-5}	Power base
1000	100	10	1	▪	1/10	1/100	1/1000	1/10000	1/100000	Value

↑
Decimal point.

The value of each digit in a number depends on the following:

- (i). The digit itself, i.e. the face value of the digit.
- (ii). The base of the number system.
- (iii). The position of the digit in the number.

Example,

Let us consider the number 8888. All the digits represent the same value of 8. However, the **positional values** are the absolute values multiplied by 10 raised to the positional power.

10^3	10^2	10^1	10^0
8	8	8	8

$$\begin{aligned}
 &= (8 \times 10^3) + (8 \times 10^2) + (8 \times 10^1) + (8 \times 10^0) \\
 &= (8 \times 1000) + (8 \times 100) + (8 \times 10) + (8 \times 1) \\
 &= 8000 + 800 + 80 + 8 \\
 &= 8888
 \end{aligned}$$

Since the positional increment is a power of 10, the value 10 is known as the **Base of the number system**.

Therefore, the **Base of a number system** is the value whose positional power is used to represent another value. Therefore, in the decimal system, the base is 10.

Example 1. To represent 5621 in the decimal system, it will be:

10^3	10^2	10^1	10^0
5	6	2	1

$$\begin{aligned}
 &= (5 \times 10^3) + (6 \times 10^2) + (2 \times 10^1) + (1 \times 10^0) \\
 &= (5 \times 1000) + (6 \times 100) + (2 \times 10) + (1 \times 1) \\
 &= 5000 + 600 + 20 + 1 \\
 &= 5621
 \end{aligned}$$

Note. In the Decimal system, the position value of each digit in a number increases 10 times as we move from right to left starting with the rightmost digit.

Example 2: Fractional numbers.

(i). 0.839

10^0		10^{-1}	10^{-2}	10^{-3}
0	.	8	3	9

$$\begin{aligned}
 &= (0 \times 10^0) + (8 \times 10^{-1}) + (3 \times 10^{-2}) + (9 \times 10^{-3}) \\
 &= (0) + (8 \times 1/10^1) + (3 \times 1/10^2) + (9 \times 1/10^3) \\
 &= (0) + (8 \times 1/10) + (3 \times 1/100) + (9 \times 1/1000) \\
 &= 0.8 + 0.03 + 0.009 \\
 &= 0.839
 \end{aligned}$$

(ii). 342.85

10^2	10^1	10^0		10^{-1}	10^{-2}
3	4	2	.	8	5

$$\begin{aligned}
 &= (3 \times 100) + (4 \times 10) + (2 \times 1) + (8 \times 1/10) + (5 \times 1/100) \\
 &= (300 + 40 + 2) + (0.8 + 0.05) \\
 &= 342.85
 \end{aligned}$$

BINARY NUMBER SYSTEM.

Binary is the representation of data by only 2 possible conditions (i.e. combinations of 1 & 0).

Binary system is a number system that uses only two digits; 0 & 1. It has a base of 2, and is therefore called a **Base-two system**.

In the binary number system, the digits '1' & '0' are referred to as **Bits (binary digits)**.

Exponential value	2^5	2^4	2^3	2^2	2^1	2^0
Integer value	32	16	8	4	2	0

It is clear that, the positional values of the numbers increase 2 times as we move from right to left. This is because the base is 2.

Points.

Bit (Binary digit) – The digit 0 or 1 in the representation of a value in Binary notation.

Binary numbers are very important in the design, organization, and understanding of computers. The Binary system is more convenient because the computer storage systems are based on a 2-state principle (digits 1 & 0).

For example;

- (i). Magnetic storage media use the magnetic principles to imitate the states of 1's & 0's. A magnetized spot represents a 1, while the non-magnetized spot represents a 0.
- (ii). The computer's Internal memory uses magnetic polarity in one direction to represent a '1' and in the reverse direction to represent a '0'.
- (iii). The computer logical operations are also affected by pulse trains, where a pulse represents a '1' and no pulse represent a '0'.

In addition, the Binary code is used only for mathematical applications (it is not intended to handle alphabetic data).

OCTAL NUMBER SYSTEM.

In Octal number system, there are only 8 possible digits (0 to 7).

Each digit (number) in base 8 has its place value determined by 8.

The position value of a digit increases to the left of the octal point in ascending powers of 8.

For example, 2163_8 can be expressed as:

$$\begin{array}{c|c|c|c} 8^3 & 8^2 & 8^1 & 8^0 \\ \hline 2 & 1 & 6 & 3 \end{array} \leftarrow \text{Assign the powers to base 8.}$$

Note. Octal number system is more popular with microprocessors, because the number represented in octal system can be used directly for input and output operations.

Complex binary numbers with several 1's and 0's can be conveniently handled in base 8. The binary digits are combined into groups of 3 (three), and each group is used to represent an individual octal digit.

HEXADECIMAL NUMBER SYSTEM.

In Hexadecimal number system, the base is 16 and there must be 16 digits.

The sixteen symbols used in the Hexadecimal system are; digits 0 to 9 & alphabets A to F.

The equivalence between hexa-numbers and decimal numbers is as shown below:-

Decimal	Hexadecimal.
10	A
11	B
12	C
13	D
14	E
15	F

Each digit in the Hex number system has its place value expressed in terms of 16.

E.g. the value 12A0 can be expressed as:

$$\begin{array}{c|c|c|c} 16^3 & 16^2 & 16^1 & 16^0 \\ \hline 1 & 2 & A & 0 \end{array} \leftarrow \text{Assign the powers to base 16.}$$

Note. Hexadecimal numbers are used because more complex binary notations can be simplified by grouping the binary digits (bits) into groups of 4's, each group representing a hexadecimal digit.

E.g. a Binary number such as 1001010100000_2 can be handled in base 16 as:

$$\begin{array}{c|c|c|c} 2^0 & 2^1 & 2^3 & 2^1 \\ \hline 0 & 0 & 1 & 0 \end{array} \leftarrow \text{Assign the powers to base 2.}$$

$$\begin{array}{l} = (1 \times 2^0) \quad (1 \times 2^1) \quad \{(1 \times 2^3) + (1 \times 2^1)\} \quad 0 \\ = 1 \quad 2 \quad (8 + 2) \quad 0 \\ = 1 \quad 2 \quad 10 \quad 0 \\ = 12A0_{16}. \end{array}$$

Note. The Hex system is very well suited for big computers such as IBM and microprocessors with CPU chips such as INTEL 8085. This is because the Hex number system represents information in short form.

Exercise I.

- State the FOUR types of Number systems.
- Define and explain Decimal notation.
- (a). What is the Binary system of numbers?
(b). Explain the main features of Binary notation.
(c). State one reason why computers use binary numbers.
- Write out what A,B,C, and D represent in the table below.

Number System	Values
A	0,1
B	0,1,2,3,4,5,6,7
C	0,1,2,3,4,5,6,7,8,9
D	0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

A _____ B _____
C _____ D _____

- (a). Explain the main characteristics of Octal notation.
(b). Explain the advantages of using Octal numbers over binary numbers.
- Write a short note on Hexadecimal notation.

BASE CONVERSIONS.

Human beings normally work with the base 10 notation, i.e. all the data passed to go as computer input is usually in decimal notation. Subsequently, the results of the computer operations are communicated to the users in a form they can understand, i.e. in base 10 (decimal) notations.

The **Base Conversion** is therefore used to help computer users understand how data and information is communicated between the computer and the user.

(i). FROM DECIMAL TO BINARY.

To convert a Decimal number (base 10) to a Binary number (base 2), the decimal number is repeatedly divided by 2, until the number cannot be further divided by two. On each division, the remainder is noted.

Then, the remainders are copied from the bottom Upwards to give the binary equivalent of the decimal number.

Example 1: Convert a decimal no. such as 121_{10} to its binary equivalent.

2	121	REM	
2	60	→	1
2	30	→	0
2	15	→	0
2	7	→	1
2	3	→	1
2	1	→	1
2	1	→	1

Copy

$$121_{10} \leftrightarrow 1111001_2$$

Example 2: Convert the following decimal integers into their binary equivalents.

(i). 36

2	36	REM	
2	18	→	0
2	9	→	0
2	4	→	1
2	2	→	0
2	1	→	0
2	1	→	1

Copy

$$36_{10} \leftrightarrow 100100_2$$

(ii). 247

2	247	REM	
2	123	→	1
2	61	→	1
2	30	→	1
2	15	→	0
2	7	→	1
2	3	→	1
2	1	→	1
2	1	→	1

Copy

$$247_{10} \leftrightarrow 11110111_2$$

Fractional numbers.

For a fractional number, the no. is divided into 2 parts; the **whole number part** & the **fraction part**.

The whole no. is then converted to binary individually as shown above. The fraction part is repetitively multiplied by 2, noting the complete units of two. This is done until the fraction becomes a 0 or starts recurring.

The complete units of the fraction part are then copied downwards.

Example 1: Convert a decimal number such as 26.25_{10} to its binary equivalent.

2	26		REM	
2	13	→		0
2	6	→	1	
2	3	→	0	
2	1	→	1	
	1	→	1	

$$26_{10} \leftrightarrow 11010_2$$

$$\text{Therefore, } 26.25_{10} \leftrightarrow 11010.01_2$$

Fractional part

$$\begin{aligned} 0.25 \times 2 &= 0.50 \rightarrow 0 \quad \text{Copy} \\ 0.50 \times 2 &= 1.00 \rightarrow 1 \\ 0.00 \times 2 &\text{ (not necessary, hence stop)} \end{aligned}$$

$$0.25 \leftrightarrow 0.01$$

(ii). FROM BINARY TO DECIMAL.

To convert a binary number to decimal system, the number in binary form is assigned weighting factors (place values) for each digit.

The **partial products** (i.e. the product of each digit and its corresponding weight) are obtained, and then added to give a decimal number equivalent of the binary number.

Example 1: Convert the binary number 10110_2 to decimal.

2^4	2^3	2^2	2^1	2^0	← Assign the powers to base 2.
1	0	1	1	0	

$$\begin{aligned} &= (1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) \\ &= 16 + 0 + 4 + 2 + 0 \end{aligned}$$

$$\text{Therefore, } 10110_2 \leftrightarrow 22_{10}$$

Explanation:

Note that the binary number 10110_2 has 5 digits. Starting with the rightmost digit, the **Most Significant Digit (MSD)** is in the 5th position. So, it is multiplied by 2^4 & each digit on its right will be half of it in its positional value. The products obtained are added together to get the required decimal equivalent.

Example 2: Convert the binary number 1111001_2 to its decimal equivalent.

2^6	2^5	2^4	2^3	2^2	2^1	2^0	← Assign the powers to base 2.
1	1	1	1	0	0	1	

$$\begin{aligned} &= (1 \times 2^6) + (1 \times 2^5) + (1 \times 2^4) + (1 \times 2^3) + (0 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\ \text{Sum} &= 64 + 32 + 16 + 8 + 0 + 0 + 1 \\ &= 121 \end{aligned}$$

$$\text{Therefore, } 1111001_2 \leftrightarrow 121_{10}$$

Example 3: Determine the decimal equivalent of the following binary numbers.

(i). 1011

Solution.

$$= (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) \\ = 8 + 0 + 2 + 1$$

Therefore, $1011_2 \leftrightarrow 11_{10}$

(ii). 11011

$$= (1 \times 2^4) + (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) \\ = 16 + 8 + 0 + 2 + 1$$

Therefore, $11011_2 \leftrightarrow 27_{10}$

(iii). 110101

$$= (1 \times 2^5) + (1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\ = 32 + 16 + 0 + 4 + 0 + 1$$

Therefore, $11010_2 \leftrightarrow 53_{10}$

(iv). 111000101

$$= (1 \times 2^8) + (1 \times 2^7) + (1 \times 2^6) + (0 \times 2^5) + (0 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\ = 256 + 128 + 64 + 0 + 0 + 0 + 4 + 0 + 1$$

Therefore, $111000101_2 \leftrightarrow 453_{10}$

Fractional numbers.

For a fractional number, the whole no. is converted to decimal as above.

The digits in the fraction part are divided by multiples of 2, starting from the decimal point.

Example 1: Convert a binary no. such as 11010.01_2 to its decimal equivalent.

2^4	2^3	2^2	2^1	2^0	.	2^{-1}	2^{-2}	Weights.
1	1	0	1	0	.	0	1	

$$= \{(1 \times 2^4) + (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (0 \times 2^0)\} \cdot \{(0 \times 2^{-1}) + (1 \times 2^{-2})\} \\ \text{Sum} = 16 + 8 + 0 + 2 + 0 + 0 + \frac{1}{4} \\ = 26 \frac{1}{4} \\ = 26.25$$

Therefore, $11010.01_2 \leftrightarrow 26.25_{10}$

Points to Note.

(i). $2^{-1} \leftrightarrow 1/2^1 \leftrightarrow \frac{1}{2}$

(ii). $5^{-2} \leftrightarrow 1/5^2 \leftrightarrow 1/25$

(iii). $2^0 \leftrightarrow 1$ (any no. raised to the power of 0 is 1.)

Example 2: Express the following binary fractions to their decimal equivalents:-

(i). $(0.111)_2$

2^0	.	2^{-1}	2^{-2}	2^{-3}	Weights.
0	.	1	1	1	

$$= (0 \times 2^0) \cdot \{(1 \times 2^{-1}) + (1 \times 2^{-2}) + (1 \times 2^{-3})\} \\ \text{Sum} = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} \\ = 0.5 + 0.25 + 0.125$$

Therefore, $0.111_2 \leftrightarrow 0.875_{10}$

(ii). $(11011.101)_2$

2^4	2^3	2^2	2^1	2^0	.	2^{-1}	2^{-2}	2^{-3}	Weights.
1	1	0	1	1	.	1	0	1	

$$\begin{aligned} &= \{(1 \times 2^4) + (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)\} \cdot \{(1 \times 2^{-1}) + (0 \times 2^{-2}) + (1 \times 2^{-3})\} \\ \text{Sum} &= 16 + 8 + 0 + 2 + 1 + \frac{1}{2} + 0 + \frac{1}{8} \\ &= 27 + 0.5 + 0.125 \end{aligned}$$

Therefore, $11011.101_2 \leftrightarrow 27.625_{10}$ **(iii). FROM DECIMAL TO OCTAL.**

To convert a Decimal number to its octal equivalent, divide the decimal number given repeatedly by 8 until the quotient obtained is zero.

Example 1: Convert 691_{10} to Octal.

8	691	REM	
8	86	→	3
8	10	→	6
8	1	→	2
8	1	→	1

Copy

Therefore, $691_{10} \leftrightarrow 1263_8$ **Example 2:** Convert the following decimal numbers to their octal equivalent.

(i). 785

8	785	REM	
8	98	→	1
8	12	→	2
8	1	→	4
8	1	→	1

Copy

Therefore, $785_{10} \leftrightarrow 1421_8$

$$\begin{aligned} \text{Check } (1421)_8 &= 1 \times 8^3 + 4 \times 8^2 + 2 \times 8^1 + 1 \times 8^0 \\ &= 512 + 256 + 16 + 1 \\ &= (785)_{10} \end{aligned}$$

(ii). 8159

8	8159	REM	
8	1019	→	7
8	127	→	3
8	15	→	7
8	1	→	7
8	1	→	1

Copy

Therefore, $8159_{10} \leftrightarrow 17737_8$

Fractional numbers.

To convert decimal fractions into their equivalent octal fractions, the whole part of the decimal number is repeatedly divided by 8.

The fractional part is repetitively multiplied by 8, noting the complete units of 8, until the fractional part becomes zero or up to the required number of digits. The complete units are then copied downwards.

Example 1: Convert a decimal number such as 98.125_{10} to its octal equivalent.

8	98	→	REM	→	
8	12	→	2	↑	
8	1	→	4	↑	Copy
8	1	→	1	↑	

$98_{10} \leftrightarrow 142_8$

Fractional part

$0.125 \times 8 = 1.00 \rightarrow 1 \downarrow$ Copy

0.00×8 (not necessary) $\downarrow$

$0.125_{10} \leftrightarrow 0.1_8$

Therefore, $98.125_{10} \leftrightarrow 142.1_8$

Example 2: Convert the following decimal fractions to their equivalent octal fractions.

(i). 0.75

$$0.75 \times 8 = 6.00$$

(Since the fractional part has become zero, there is no need to proceed further).

$$\text{Therefore, } 0.75_{10} \leftrightarrow 0.6_8$$

$$\text{Check } (0.6)_8 = 6 \times 8^{-1} = 6 \times (1/8) = 6 \times 0.125 = (0.75)_{10}$$

(ii). 0.21875

$$\begin{array}{l} 0.21875 \times 8 = 1.75000 \rightarrow 1 \downarrow \text{Copy} \\ 0.75000 \times 8 = 6.00000 \rightarrow 6 \downarrow \end{array}$$

$$\text{Therefore, } 0.21875_{10} \leftrightarrow 0.16_8$$

$$\begin{aligned} \text{Check } (0.16)_8 &= 1 \times 8^{-1} + 6 \times 8^{-2} \\ &= 1 \times (1/8) + 6 \times (1/64) \\ &= (1 \times 0.125) + (6 \times 0.015625) \\ &= 0.125 + 0.093750 \\ &= (0.21875)_{10} \end{aligned}$$

(iv). FROM OCTAL TO DECIMAL.

The octal number is assigned the weights in terms of 8's.

Example 1: Convert 1263_8 to decimal.

8^3	8^2	8^1	8^0	← Assign the powers to base 8.
1	2	6	3	

$$\begin{aligned} \text{Sum} &= (1 \times 8^3) + (2 \times 8^2) + (6 \times 8^1) + (3 \times 8^0) \\ &= 512 + 128 + 48 + 3 \\ &= 691 \end{aligned}$$

$$\text{Therefore, } 1263_8 \leftrightarrow 691_{10}$$

Example 2: Convert the following octal numbers to their decimal equivalents.

(i). 41

$$(41)_8 = (4 \times 8^1) + (1 \times 8^0)$$

$$\begin{aligned} \text{Sum} &= 32 + 1 \\ &= 33 \end{aligned}$$

Therefore, $(41)_8 \leftrightarrow (33)_{10}$

(ii). 283

$$(283)_8 = (2 \times 8^2) + (8 \times 8^1) + (3 \times 8^0)$$

$$\begin{aligned} \text{Sum} &= 128 + 64 + 3 \\ &= 195 \end{aligned}$$

Therefore, $(283)_8 \leftrightarrow (195)_{10}$

Example 3: Convert $6057_8 \times 42_8$ to decimal.

8^3	8^2	8^1	8^0	← Assign the powers to base 8.
6	0	5	7	

$$\begin{aligned} &= (6 \times 8^3) + (0 \times 8^2) + (5 \times 8^1) + (7 \times 8^0) \\ \text{Sum} &= 3072 + 0 + 40 + 7 \\ &= 3119_{10} \end{aligned}$$

8^1	8^0	← Assign the powers to base 8.
4	2	

$$\begin{aligned} &= (4 \times 8^1) + (2 \times 8^0) \\ \text{Sum} &= 32 + 2 \\ &= 34_{10} \end{aligned}$$

$$\begin{array}{r} 3119_{10} \\ \times 34_{10} \\ \hline 9357 \\ 12476 \\ \hline 106046 \end{array}$$

Therefore, $6057_8 \times 42_8 \leftrightarrow 106046_{10}$

Fractional numbers.

For a fractional octal number, the whole part is converted to decimal as above. The digits in the fraction part are divided by multiples of 8.

Example 1: Convert 142.1_8 to decimal.

8^2	8^1	8^0		8^{-1}	← Assign the powers to base 8
1	4	2	.	1	

$$\begin{aligned} &= \{(1 \times 8^2) + (4 \times 8^1) + (2 \times 8^0)\}. (1 \times 8^{-1}) \\ \text{Sum} &= 64 + 32 + 2 + 1/8 \\ &= 98 \frac{1}{8} \\ &= 98.125 \end{aligned}$$

Therefore, $142.1_8 \leftrightarrow 98.125_{10}$

Example 2: Convert the following octal fractions to their decimal equivalents.

(i). $(0.54)_8$

$$\begin{aligned}(0.54)_8 &= (5 \times 8^{-1}) + (4 \times 8^{-2}) \\ &= 5 \times (1/8) + 4 \times (1/64) \\ \text{Sum} &= 5 \times 0.125 + 4 \times 0.015625 \\ &= 0.625 + 0.062500 \\ &= 0.687500\end{aligned}$$

Therefore, $(0.54)_8 \leftrightarrow (0.687500)_{10}$

(ii). $(0.642)_8$

$$\begin{aligned}(0.642)_8 &= (6 \times 8^{-1}) + (4 \times 8^{-2}) + (2 \times 8^{-3}) \\ &= 6 \times (1/8) + 4 \times (1/64) + 2 \times (1/512) \\ \text{Sum} &= 6 \times 0.125 + 4 \times 0.015625 + 2 \times 0.001953125 \\ &= 0.750 + 0.062500 + 0.003906250 \\ &= 0.816406250\end{aligned}$$

Therefore, $(0.642)_8 \leftrightarrow (0.816406250)_{10}$

(v). FROM DECIMAL TO HEXADECIMAL.

The decimal no. is repeatedly divided by 16.

Example 1: Convert 122_{10} to its hexadecimal equivalent.

16	122	→	REM	
16	7	→	A (=10 in decimal)	↑ Copy
	7	→	7	

Therefore, $122_{10} \leftrightarrow 7A_{16}$

Example 2: Convert decimal number 745 to equivalent hexadecimal number.

16	745	→	REM	
16	46	→	9	↑ Copy
16	2	→	E (=14 in decimal)	
	2	→	2	

Therefore, $745_{10} \leftrightarrow 2E9_{16}$

Fractional numbers.

For a fractional decimal number, the fraction part is repetitively multiplied by 16, noting the complete units of 16's. The complete units are then copied downwards.

Example 1: Convert 32.125_{10} to hexadecimal.

16	32	→	REM		
16	2	→	0	↑ Copy	Fractional part
	2	→	2		0.125 × 16 = 2.00 → 2
					0.00 × 16 (not necessary) ↓

$32_{10} \leftrightarrow 20_{16}$

$0.125_{10} \leftrightarrow 0.2_{16}$

Therefore, $32.125_{10} \leftrightarrow 20.2_{16}$

(vi). FROM HEXADECIMAL TO DECIMAL.

Assign the hexadecimal number given to the weights in terms of 16's.

Example 1: Convert $7A_{16}$ to decimal.

16^1	16^0	$\leftarrow$ Assign the powers to base 16.
7	A	

$$\begin{aligned} &= (7 \times 16^1) + (A \times 16^0) \\ \text{Sum} &= 112 + 10 \\ &= 122 \end{aligned}$$

Therefore, $7A_{16} \leftrightarrow 122_{10}$

Fractional numbers.

For a fractional hexadecimal value, the whole part is converted to decimal as above. The digits in the fraction part are divided by multiples of 16.

Example 1: Convert 20.2_{16} to decimal.

16^1	16^0		16^{-1}	$\leftarrow$ Assign the powers to base 16.
2	0	.	2	

$$\begin{aligned} &= \{(2 \times 16^1) + (0 \times 16^0)\} + (2 \times 16^{-1}) \\ \text{Sum} &= 32 + 0 + (2 \times 1/16) \\ &= 32 + 1/8 \\ &= 32 \frac{1}{8} \\ &= 32.125 \end{aligned}$$

Therefore, $20.2_{16} \leftrightarrow 32.125_{10}$

Example 2: Convert the following hexadecimal fractions into their decimal equivalents.

(i). $(0.8)_H$

$$\begin{aligned} (0.8)_H &= 8 \times 16^{-1} \\ &= 8 \times (1/16) \\ &= 0.5000 \end{aligned}$$

Therefore, $(0.8)_H = (0.5000)_{10}$

(ii). $(A9.328)_H$

$$\begin{aligned} (A9.328)_H &= \{A \times 16^1 + 9 \times 16^0\} + \{3 \times 16^{-1} + 2 \times 16^{-2} + 8 \times 16^{-3}\} \\ &= 10 \times 16 + 9 \times 1 + 3 \times (1/16) + 2 \times (1/256) + 8 \times (1/4096) \\ &= 160 + 9 + 3 \times 0.0625 + 2 \times 0.00390625 + 8 \times 0.00024414 \\ &= 160 + 9 + 0.1875 + 0.0078125 + 0.001953125 \\ &= 169.197265625 \end{aligned}$$

Therefore, $(A9.328)_H = (169.197265625)_{10}$

(vii). FROM BINARY TO OCTAL.

Example 1: Convert the binary number 10001110011_2 to Octal.

Solution.

The binary digits are grouped into 3's and each group is used to represent an individual octal digit.

2^1	2^0	2^2	2^1	2^1	2^0	$\leftarrow$ Assign the powers to base 2.
0	1	0	0	1	1	

$$\begin{aligned}
 &= (1 \times 2^1) \quad (1 \times 2^0) \quad \{(1 \times 2^2) + (1 \times 2^1)\} \quad \{(1 \times 2^1) + (1 \times 2^0)\} \\
 &= \quad 2 \quad \quad 1 \quad \quad (4 + 2) \quad \quad (2 + 1) \\
 &= \quad 2 \quad \quad 1 \quad \quad \quad 6 \quad \quad \quad 3 \\
 &= 2163_8.
 \end{aligned}$$

(viii). FROM BINARY TO HEXADECIMAL.

Example: Convert the binary number 0110110010001111 to its equivalent hexadecimal number.

Solution.

The binary digits are put into groups of 4's, each group representing a hexadecimal digit.

2^2	2^1	2^3	2^2	2^3	2^3	2^2	2^1	2^0	
0	1	1	0	1	1	0	0	0	1

$$\begin{aligned}
 &= \{(1 \times 2^2) + (1 \times 2^1)\} + \{(1 \times 2^3) + (1 \times 2^2)\} + \{(1 \times 2^3)\} + \{(1 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)\} \\
 &= \{4 + 2\} \quad \{8 + 4\} \quad \{8\} \quad \{8 + 4 + 2 + 1\} \\
 &= 6 \quad \quad 12 \quad \quad 8 \quad \quad 15 \\
 &= 6C8F_{16}.
 \end{aligned}$$

The equivalent hexadecimal number is $(6C8F)_{16}$.

NOTE. You can convert an Octal number to Binary, Hexadecimal to Binary or from Octal to Hexadecimal.

To do this, first convert the given number to base 10 (decimal), then from base 10 to the required base.

(ix). FROM OCTAL TO BINARY.

Example1: Convert 347_8 to Binary.

Solution.

Step 1: First, convert the given number to base 10 (decimal).

8^2	8^1	8^0	← Assign the powers to base 8.
3	4	7	

$$\begin{aligned}
 &= (3 \times 8^2) + (4 \times 8^1) + (7 \times 8^0) \\
 &= (3 \times 64) + (4 \times 8) + (7 \times 1) \\
 &= 192 + 32 + 7 \\
 &= 231_{10}.
 \end{aligned}$$

Step 2: Then convert the resulting decimal number to the binary.

2	231	REM	
2	115	→	1
2	57	→	1
2	28	→	1
2	14	→	0
2	7	→	0
2	3	→	1
2	1	→	1
	1	→	1

Copy

$231_{10} \leftrightarrow 11100111_2$

Therefore, $347_8 \leftrightarrow 11100111_2$

(x). FROM HEXADECIMAL TO BINARY.**Example 1:** Convert $6DC_{16}$ to its binary equivalent.**Solution.***Step 1:* Convert the hexadecimal (base 16) number given to its decimal (base 10) equivalent. $6DC$

$$\begin{aligned}
 &= (6 \times 16^2) + (D \times 16^1) + (C \times 16^0) \\
 &= 6 \times 16^2 + 13 \times 16^1 + 12 \times 16^0 \\
 &= 1536 + 208 + 12 \\
 &= 1756_{10}.
 \end{aligned}$$

Step 2: Convert the decimal (base 10) number obtained to binary (base 2).

2	1756	REM	
2	878	→	0
2	439	→	0
2	219	→	1
2	109	→	1
2	54	→	1
2	27	→	0
2	13	→	1
2	6	→	1
2	3	→	0
2	1	→	1
2	1	→	1

Therefore, $(6DC)_{16} \leftrightarrow (11011011100)_2$ **Example 2:** Convert $A71_{16}$ to binary.**Solution.***Step 1:* Convert the hexadecimal (base 16) number given to its decimal (base 10) equivalent. $A71$

$$\begin{aligned}
 &= (A \times 16^2) + (7 \times 16^1) + (1 \times 16^0) \\
 &= 10 \times 16^2 + 7 \times 16^1 + 1 \times 16^0 \\
 &= 2560 + 112 + 1 \\
 &= 2673_{10}.
 \end{aligned}$$

Step 2: Convert the decimal (base 10) number obtained to binary (base 2).

2	2673	REM	
2	1336	→	1
2	668	→	0
2	334	→	0
2	167	→	0
2	83	→	1
2	41	→	1
2	20	→	1
2	10	→	0
2	5	→	0
2	2	→	1
2	1	→	0
2	1	→	1

Therefore, $(A71)_{16} \leftrightarrow (101001110001)_2$

(xi). FROM OCTAL TO HEXADECIMAL.**Example1:** Convert 6057_8 to Hexadecimal.**Solution.****Step 1:** First, convert the given octal number to decimal (base 10).

8^3	8^2	8^1	8^0	← Assign the powers to base 8.
6	0	5	7	

$$\begin{aligned}
 &= (6 \times 8^3) + (0 \times 8^2) + (5 \times 8^1) + (7 \times 8^0) \\
 \text{Sum} &= 3072 + 0 + 40 + 7 \\
 &= 3119
 \end{aligned}$$

Therefore, $6057_8 \leftrightarrow 3119_{10}$ **Step 2:** Then convert the resulting decimal number to the Hexadecimal.

16	3119	→	REM	
16	194	→	F (=15 in decimal)	↑ Copy
	12	→	2	
	12	→	C (=12 in decimal)	

$$3119_{10} \leftrightarrow C2F_{16}$$

Therefore, $6057_8 \leftrightarrow C2F_{16}$ **Exercise I.****1. (A).** What are Base Conversions?**(B).** Explain the following conversions:

- (i).** From decimal to binary.
- (ii).** From binary to decimal.
- (iii).** From decimal to Octal.
- (iv).** From octal to decimal.
- (v).** From decimal to hexadecimal.
- (vi).** From hexadecimal to decimal.

2. Perform the following computer arithmetic. In each case, show how you arrive at your answer.**(A).** Convert the following decimal numbers to their binary equivalent.

- (i).** 11
- (ii).** 001
- (iii).** 255
- (iv).** 2346
- (v).** 322.6875 (3 marks).

(B). Convert the following binary numbers to decimal.

- (i).** 110111001 (2 marks)
- (ii).** 10101.1011 (2 marks)

(C). Convert the following binary numbers to hexadecimal number systems.

- (i).** 1011 (2 marks).
- (ii).** 11110101 (2 marks).
- (iii).** 10100000001111111
- (iv).** 110111.11 (2 marks).
- (v).** 1.1110101 (2 marks).
- (vi).** 0.10111010110011101111

- (D). Convert the following octal numbers to binary.
- (i). 77
 - (ii). 0000001
 - (iii). 347
 - (iv). 1263
 - (v). 473.4 (2 marks).
 - (vi). 0.354672341
- (E). Convert the following hexadecimal numbers to binary.
- (i). 13
 - (ii). C3
 - (iii). 347.2
 - (iv). BCD
 - (v). A71
 - (vi). A71.A
 - (vii). 0.8746532
 - (viii). 0.7BCEFAD
- (F). Convert the following hexadecimal numbers to decimal. (2 marks).
- (i). E1F
- (G). Convert the following hexadecimal numbers to octal. (4 marks).
- (i). F8
 - (ii). FBE.E1AB
- (H). Convert the following decimal numbers to hexadecimal. (2 marks).
- (i). 32.125
 - (ii). 5.125
- (I). Convert the following binary numbers to octal.
- (i). 0.00101110010111
 - (ii). 0.101110011111010001

Exercise II.

1. Convert the following to Hexadecimal.
 - (i). 240_{10} (2 marks).
 - (ii). 6057_8 (4 marks).
2. (A). Convert the following numbers to their Binary equivalents.
 - (i). 376_8 . (3 marks).
 - (ii). $A6B_{16}$. (4 marks).
 (B). Convert the following into binary:
 - (i). 347_8 .
 - (ii). 473_{10} .
 - (iii). BCD_{16} . (12 marks).
3. A. Convert the following numbers into their decimal equivalent showing all steps.
 - (i). 110101.0110_2
 - (ii). 347_8
 B. Convert the following numbers into their hexadecimal equivalent showing all steps
 - (i). 356_{10}
 - (ii). 6754_8
4. Convert the following octal numbers to hexadecimal.
 - (i). 6057
 - (ii). 56752 (3 marks).

NEGATIVE NUMBERS.

Both positive and negative numbers can be represented in the computer memory during processing.

The input numbers whether positive or negative are all represented in binary form.

The negative numbers are used to carry out subtraction in the computer's arithmetic operations. This is based on the fact that, subtracting a number is the same as adding its negative to the other.

The following are the various methods used to represent negative numbers in the computer:-

- (i). One's Complement (1C) method.
- (ii). Two's Complement (2C) method.
- (iii). Signed Magnitude method.

One's Complement (1C) method.

In 1C method, the binary digits representing the negative number are negated, (i.e., the 1's in the number are changed to 0's, and the 0's are changed to 1's).

E.g., -17_{10} can be represented in binary as a negative value as:

Step 1: Convert -17_{10} to binary (base 2).

2	17	→	REM	
2	8	→	1	↑ Copy
2	4	→	0	
2	2	→	0	
2	1	→	0	
1		→	1	

$17_{10} \leftrightarrow 10001_2$

Therefore, $-17_{10} \rightarrow 1\ 0\ 0\ 0\ 1_2$

↓	↓	↓	↓	↓	
0	1	1	1	0	Complementing each bit

$-17_{10} \leftrightarrow 01110_2$ (1C method).

Two's Complement (2C) method.

In 2C method, the negative number is represented into binary form, then complemented as in 1C method, but a '1' is added to the Least Significant Digit (LSD) of the complement value.

E.g., -17_{10} can be represented as:

$17_{10} \leftrightarrow 10001_2$

Therefore, $-17_{10} \rightarrow 1\ 0\ 0\ 0\ 1_2$

↓	↓	↓	↓	↓	
0	1	1	1	0	Complement each bit, add a '1' to the complement.
1					
0	1	1	1	1	

$-17_{10} \leftrightarrow 01111_2$ (2C method).

Signed Magnitude method.

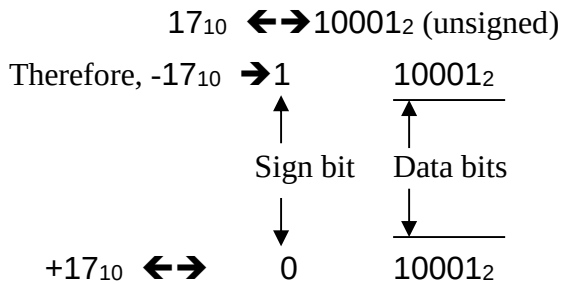
In Signed Magnitude method, the number is divided into two parts; the **Sign part** and the **Data** part.

Usually, a '1' is used to represent a negative sign and a '0' to represent a positive sign.

The Magnitude part is expressed in binary but not complimented. Therefore, one part of the binary digits will be representing the data value, while the other will represent the sign bit.

To make it simple, it is assumed that the sign bit is 1, while the data bits are equivalent to the binary number digits representing 17.

E.g., -17_{10} can be represented in binary as 10001_2 :

**Exercise.**

1. (a). What are Negative numbers?
- (b). Explain the main methods of using negative numbers.

BINARY ARITHMETICS.**ADDITION:**

Adding binary numbers is the same as adding decimal numbers. The difference is that, any complete units of two's are carried and added to the next significant digit.

Binary addition rules.

During binary addition (i.e. when adding two digits) the following rules need to be noted.

				Sum	Carry
0	+	0	=	0	0
0	+	1	=	1	0
1	+	0	=	1	0
1	+	1	=	0	1

Example 1:

$$\begin{array}{r}
 1 \ 0 \ 0 \ 1 \\
 + \ 0 \ 1 \ 0 \ 1 \\
 \hline
 1 \ 1 \ 1 \ 0
 \end{array}$$

Example 2:

$$\begin{array}{r}
 1 \ 1 \ 0 \ 1 \ 0_2 \\
 + \ 1 \ 0 \ 1 \ 0_2 \\
 \hline
 1 \ 0 \ 0 \ 1 \ 0 \ 0_2
 \end{array}$$

Explanation:

Addition of two binary numbers starts from the Least Significant Digits.

- ♦ Zero (0) is added to 0 to get a sum of 0.

- ♦ The next significant digits, 1 and 1 are added to get a sum of 0 plus a carryout of 1. The **Carryout** is given by any complete units of two got after the digits of the same significance are added.
The carryout from the previous addition becomes the Carry in of the next significant digits, to which the carry is added.
The procedure continues until the overall sum of the two binary numbers is got.

Note. During arithmetic operations, the data to be worked upon are usually held on locations (cells) of predefined length or capacity. If the results cannot fit in any of the spaces that contain the values to be executed, then there is usually an excess bit in the answer.

For instance, in the above example, we have added a 5-bit and a 4-bit number, to get a 6-bit answer, which cannot fit on either 5-bit or 4-bit areas, hence there is an excess bit in the answer, what is described as the **Arithmetic overflow**.

The excess/extra bits of the sum that cannot fit the expected space are known as **Overflow**.

SUBTRACTION:

Binary subtraction rules.

When subtracting two numbers (i.e. when getting the difference between two numbers), the following rules must be observed.

				Difference	Borrowed (from the next significant digit)
0	-	0	=	0	0
1	-	0	=	1	0
0	-	1	=	1	1
1	-	1	=	0	0

Example 1:

$$\begin{array}{r}
 1\ 1\ 1\ 0\ 1_2 \\
 -\ 1\ 0\ 1\ 0_2 \\
 \hline
 1\ 0\ 0\ 1\ 1_2
 \end{array}$$

NB. The one (1) you have borrowed is equivalent to 2.

Explanation:

The subtraction starts with the least significant digits towards the most significant digits.

- ♦ In the first subtraction, a 0 is subtracted from a 1, and the difference is a '1'.
- ♦ In the 2nd subtraction, 1 cannot be subtracted from a 0, and hence a 1 containing 2 is borrowed from the next significant digit from which a 1 is subtracted and a difference of 1 is got.
Note that, borrowing 1 from 1 leaves a 0 in the 3rd digit of the number being subtracted from.
The procedures proceed until the subtraction is complete.

Example 2:

11110011 – 1100000 (base 2).

$$\begin{array}{r}
 11110011 \\
 -\ 1100000 \\
 \hline
 10010011
 \end{array}
 \quad (\text{base } 2)$$

Note. Subtraction of binary numbers can also be carried out using the Compliments and Signed magnitude methods.

Subtraction of binary numbers using the One's Complement method.

$$(11101)_2 - (1010)_2$$

Solution.

Step 1: Make the digits of the values the same.

$$\begin{array}{r} 1\ 1\ 1\ 0\ 1 \\ -\ 0\ 1\ 0\ 1\ 0 \\ \hline \end{array}$$

Step 2: Complement the digits of the negative number, and then add.

$$\begin{array}{r} 1\ 1\ 1\ 0\ 1 \\ +\ 1\ 0\ 1\ 0\ 1 \\ \hline \underline{1\ 1\ 0\ 0\ 1\ 0} \end{array}$$

↑
Overflow

Step 3: Add the overflow digit to the Least Significant Digit of the sum to get the difference.

$$\begin{array}{r} 1\ 1\ 1\ 0\ 1 \\ +\ 1\ 0\ 1\ 0\ 1 \\ \hline 1\ 0\ 0\ 1\ 0 \\ 1\ + \\ \hline 1\ 0\ 0\ 1\ 1 \leftarrow \text{Difference} \end{array}$$

MULTIPLICATION:

When two numbers are multiplied, the output is the **Product**.

Binary multiplication rules.

The following rules must be used when carrying out a multiplication procedure.

Product				
0	x	0	=	0
0	x	1	=	0
1	x	0	=	0
1	x	1	=	1

Example 1:

$$\begin{array}{r} 1\ 0\ 1\ 0 \leftarrow \text{Multiplicand} \\ \times\ 1\ 0\ 1 \leftarrow \text{Multiplier} \\ \hline 1\ 0\ 1\ 0 \\ 0\ 0\ 0\ 0 \\ 1\ 0\ 1\ 0 \\ \hline 1\ 1\ 0\ 0\ 1\ 0 \leftarrow \text{Product} \end{array}$$

} ← partial products

Explanation:

The multiplicand is multiplied by the multiplier digits starting from the Most Significant Digit towards the least significant digit, shifting one place to the right each time the next multiplier digit is used to produce the partial product.

The partial products are then added up to give the product of the multiplication.

Example 2:

- (i).
- 11100×10010
- (base 2).

$$\begin{array}{r}
 11100 \\
 \underline{10010} \times \\
 11100 \\
 00000 \\
 00000 \\
 11100 \\
 \underline{00000} \\
 \underline{111111000} \text{ (base 2)}
 \end{array}$$

DIVISION:

Computers carry out binary division by repetitive subtraction.

Example 1:

$$\begin{array}{r}
 1110_2 \text{ REM } 100_2 \\
 101 \overline{) 1001010} \\
 \underline{-101} \\
 1000 \\
 \underline{-101} \\
 0111 \\
 \underline{-101} \\
 0100
 \end{array}$$

Explanation:

Since the divisor is made up of 3 digits, we start by grouping the dividend bits into groups of 3's from the Most Significant Digit.

- ♦ In the first grouping, 100 is got, which cannot be divided by 101, and hence extend to 4 digits and 101 goes into 1001 once. The 1 is copied on top.
The remainder 100 cannot divide into 101, and hence the next digit 0 is added to it forming 1000, that divides into 101 once, and 1 is copied on top.
The procedure repeats until all the digits of the dividend are exhausted and a remainder of 1110_2 is got.

Example 2:

$$\begin{array}{r}
 110 \\
 101 \overline{) 11110} \\
 \underline{101} \\
 101 \\
 \underline{101} \\
 00 \\
 \rightarrow 110
 \end{array}$$

Question.

Perform the following computations:

- (i). 11100×10010 (base 2).
 (ii). $2567 + 3476$ (base 8).

$$\begin{array}{r}
 2567 \\
 \underline{3476} + \\
 \underline{6265} \text{ (base 8)}
 \end{array}$$

- (iii). $11110011 - 1100000$ (base 2).

Exercise I.

1. Explain the following processes of binary arithmetic:
 - (i). Addition.
 - (ii). Subtraction.
 - (iii). Multiplication.
 - (iv). Division.
2. Use One's complement to solve the following sums:
 - (i). 9-6
 - (ii). 17-15
3. Use Two's complement to solve the following sums (the numbers are in decimal notation).
 - (a). 13-10.
 - (b). 23-20
 - (c). 17-14
4. Carry out the following arithmetic:
 - (i). 110111_2 Multiply by 1101_2
 - (ii). 6057_8 multiply by 42_8 (3 marks).
 - (iii). 2.4_8 multiply by 7.6_8 (3 marks).
 - (iv). 110110_2 divide by 111_2 (3 marks).
5. Use binary addition to solve the following decimal summations.
 - (i). 4+3
 - (ii). 13+2
6. Perform the following binary additions. Your answers should be in the binary format:
 - (i). $1010 + 0010$
 - (ii). $1111 + 0001$

Exercise II.

1. Perform the following binary arithmetic:
 - (A). (i). 10111 multiply by 1001
(ii). 1011.1 multiply by 1.01 (3 marks)
 - (B). (i). 1110011 divide by 101
(ii). 11100.11 divide by 1.01 (3 marks)
 - (C). (i). 1110.0111 add 1101001.0110
(ii). 11101101 add 0101110 (2 marks)
 - (D). 1001011.011 subtract 111.111 (2 marks).
2. Perform the following hexadecimal arithmetic:
 - (a). $31 - 9.7_{FD}$ (3 marks).
 - (b). $ABCD + 145$ (3 marks).
 - (c). $DFAC - 101B$ (2 marks).
 - (d). $14.79 + BC.78 + DE.78$ (2 marks).
3. Perform the following octal arithmetic:
 - (i). $2.456 + 17.456 + 31.76$ (2 marks).
 - (ii). 57724.33 add 422.76 (2 marks).
 - (iii). 20 multiply by 13 (2 marks).
 - (iv). 243.4 multiply by 123 (3 marks).
 - (v). 421×4.7 (3 marks).
 - (vi). 124 divide by 10 (2 marks).
 - (vii). 260032 divided by 246 (3 marks).
 - (viii). $21.256 - 6.347$ (2 marks).
 - (ix). 6625.24 subtract 751.76 (2 marks).

COMPUTATIONAL ERRORS.

The main computation errors are:

1). Overflow:

Overflow occurs whenever the results of an arithmetic operation are bigger than the assigned work area.

E.g. If the result is to be stored in an 8-bit work area (register) and the results generate more than 8 bits, an error message is generated by the computer. This may terminate the normal execution cycle.

2). Truncation:

To **truncate a number** is to drop out some digits of the number after the decimal point when for example a division or fraction gives a recurring quotient.

E.g. Division of $1/9$ produces $0.1111\dots$

The quotient, therefore, may be rounded down to 3 decimal places, thereby dropping out the rest of the values.

I.e. $1/9 = 0.111$

3). Rounding Up.

If a division produces a recurring quotient or in fractions that recur, the result (quotient) can be rounded to the nearest value.

E.g., $23/48 = 0.4791666\dots$

In this case, the quotient can be rounded up to 4 decimal places as 0.4792 , because the next value after the 4th decimal place is ≥ 5 . However, if it was less than 5, then it would have been rounded down to 0.4791 .

The rounding of numbers (answers/quotients) reduces truncation errors.

4). Floating Point Numbers.

The **floating point notation** is the expression of a number or values into their standard form equivalent.

E.g., 1234 can be expressed in standard (floating point) notations as:

$$1.234 \times 10^3$$

Where; 1.234 is the **Argument**.

10 is the **Base**.

3 is the **Exponent**.

Calculations done in floating point notations or any numbers represented in their floating notation are prone to truncation errors. For example, to represent a number such as 0.1234×10^4 into a 3-digit accuracy, then the result is truncated.

However, computers with larger storage areas for floating point numbers are less prone to such truncation errors.

Exercise.

1. Name and explain the four types of computation errors. (20 marks).

COMPUTER CODING SYSTEMS.

The decimal number system is the accepted international system. However, computers can understand and work only using the binary language, i.e., in 0's & 1's.

Therefore, in order to talk to computers, one has to convert the information (whether numeric or non-numeric) into binary form. The process of representing numeric or non-numeric information in terms of binary digits is known as **Coding**.

By using different bit patterns, the non-numeric information, such as letters of the English alphabet A-Z can be represented.

E.g., letter A is represented as: 110001, while the equal (=) sign is represented as 001011.

In addition, the Digital computers that are frequently used in many business environments use binary digits to operate, i.e., they accept data & instructions in terms of 1's and 0's, work upon them and produce information, which is immediately stored in the form of 0's & 1's before they are communicated to the end users.

This means that coding in computers is done to achieve the following objectives:

- 1). To convert numeric & alphanumeric information into binary digits of 0 & 1.
- 2). To transmit correct data to the computer.
- 3). To detect and correct errors, if any, during the transmission of data.

The following are the various methods of representing data in Binary Number systems in a computer:

1. BCD (Binary Coded Decimal).
2. EBCDIC (Extended Binary Coded Decimal Interchange Code).
3. ASCII (American Standard Code for Information Interchange) Code.

What is a Code?

- ❖ A set of rules, outlining the way in which data may be represented.
- ❖ Rules used to convert data from one representation to another, e.g. from ASCII to EBCDIC.

BCD Code.

BCD stands for **Binary Coded Decimal**.

Usually, there are 2 types of BCD coding;

- ❖ 4-bit BCD Coding.
- ❖ Extended BCD Code.

(1). 4-bit BCD Coding.

This method uses 4 binary digits to represent an individual decimal digit.

Example;

3	9	8 ₍₁₀₎
↓	↓	↓
0011	1001	1000

A group of 4 bits (binary digits) usually representing a number is known as a **NIBBLE**.

Decimal	BCD	Pure Binary
398 ₁₀	001110011000	110001110 ₂

Note. The BCD notation of the number 398₁₀ is not the same as when represented in the pure binary notation.

The BCD notation uses more bits than the pure binary notation, and is arithmetically slower. However, the BCD operations are simple, they remove rounding (truncation) errors & are used to store data in packed mode.

(2). Extended BCD Code.

This is a 6-bit coding system. It consists of 4 **data/numeric bits**, and 2 additional bits known as the **Zone bits**. An extra bit known as the **Parity bit** is added (7 bits, when the parity bit is included).

The **Zone bits** indicate the type of characters represented by the code, while the **Data bits** represent the particular characters represented by the codes.

Numerals, alphabets and special characters can be represented in their binary form using the Extended BCD coding system. The characters are classified into zones, which are then assigned zone bits.

Example 1: Zoning of the characters A – Z.

Zone	Characters	Zone bits
1	A – I	11
2	J – R	10
3	S – Z	01

Counting starts from the 1st character within a zone and ends at the last character within that zone.

Note. The **Zone bits**, in computer with numeric bits, form codes to present characters.

Example 2: The Numeric equivalents of the characters within the zone A – I.

Zone A – I characters	Numeric value	Binary
A	1	0001
B	2	0010
C	3	0011
D	4	0100
.	.	.
.	.	.
.	.	.
I	9	1001

Similarly, the other numeric values of the characters in the other zones can be determined, i.e., J=1 to R=9 and S=2 to Z=9.

The Extended BCD uses unique combinations of two zero bits and 4 numeric bits to represent specific data characters.

To represent a given character in Extended Binary code, the zone bits are added to the **data bits** (the binary digits representing the numerical equivalent to the character), and an extra bit (the **parity bit**) added.

To represent a character like A in Extended BCD code:

The zone bits for A are '11' and data bits of A are '0001'.

X	11	0001
↓	↓	↓
Parity bit	Zone bits	Data bits

The Parity bit 'x' depends on the machine/computer in use, i.e., whether **Even** or **Odd parity** machine.

For an **Even parity machine**, the no. of 1's in a code representing a character adds up to an even number, while for an **Odd parity machine**, the no. of 1's in a code representing a character adds up to an odd number.

Therefore, the value of 'x' is '1' for an even parity machine and '0' for an odd parity machine.

Note that, if X=1, then the no. of 1's in the code for A are 4, which is an even number. If X=0, then the number of 1's remain 3, which is an odd no.

Parity bit – A check bit appended to a collection/group of binary digits to make the sum of all the binary digits, including the check bit, always odd or always even.

The Parity bit does not change the value of the character stored, but it is used to automatically detect transmission errors during data communication, e.g., from an external storage media like cassette to the internal memory. If some bits fall off, the computer responds with a parity failure error message.

Parity Check – A check to determine whether the number of bits in a pattern is either even or odd.

TYPICAL CODING SYSTEM FOR STANDARD BCD CODE.

0 - 000000	A - 010000	L - 100010	W - 110100
1 - 000001	B - 010001	M - 100100	X - 110101
2 - 000010	C - 010010	N - 100101	Y - 110110
3 - 000011	D - 010011	O - 100110	Z - 110111
4 - 000100	E - 010100	P - 100110	
5 - 000101	F - 010101	Q - 100111	
6 - 000110	G - 010110	R - 101000	
7 - 000111	H - 010111	S - 110000	
8 - 001000	I - 011000	T - 110001	
9 - 001001	J - 100000	U - 110010	
	K - 100001	V - 110011	

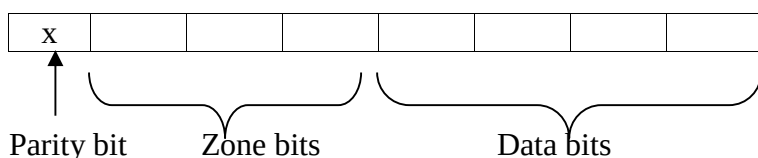
ASCII Code.

ASCII stands for **American Standard Code for Information Interchange**.

This is a standard 7-bit coding system, with 3-zone bits added to the 4-data bits, and an extra parity bit (8 bits when the parity bit is included).

ASCII code is used for data interchange involving computers and communications equipment.

The ASCII code format for a given character therefore looks like:



E.g., letter A can be expressed as; X1011001.

Note. The ASCII coding system can represent up to 128 characters, i.e., 2^7 characters.

EBCDIC Code.

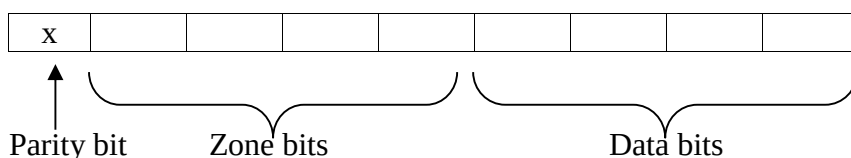
EBCDIC stands for **Extended Binary Coded Decimal Interchange Code**.

This is an 8-bit coding system, with 4 zone bits, 4 data bits and an extra parity bit (in total it consists of 9 bits, when the parity bit is included).

The EBCDIC coding system is an extension of the 6-bit BCD coding system. It allows for the representation of more characters, i.e., 256 (2^8) characters as compared to the 64 (2^6) characters that can be represented in the 6-bit BCD coding system.

The EBCDIC uses unique combinations of 4 zone bits & 4 numeric bits to represent specific data characters.

The EBCDIC code format for a given character would look like:



Exercise I.

1. (a). What is computer coding systems? (5 marks).
- (b). Name and briefly explain TWO computer coding systems. (4 marks).
- (c). State advantages of using Extended BCD code over 4-bit BCD code. (6 marks).
- (d). Explain the purpose of zone bits, and parity bit in a computer coding system. (4 marks).

Exercise II.

1. (a). What is meant by the term Character codes? Give examples.
- (b). Explain the principal characteristics of the following by the help of suitable examples:
 - (i). 4-bit BCD code.
 - (ii). Parity bit.
 - (iii). ASCII code.
 - (iv). EBCDIC code.

Exercise III.

1. State THREE methods of representing data in Binary number system.

STORAGE ORGANISATION.

The computer memory is designed in such a way that the basic unit of the stored entities, i.e., data, instructions and information, is a 2-state unit representing a '1' or '0'. The 2-state quantity is described as a **Bit**, **Cell** or **Binary digit**. In other words, storage is only possible in terms of 0's & 1's.

The two units can be described by electric circuit using two voltages. When there is voltage (when the circuit is on), a '1' is represented, and when there is no voltage in the circuit (when it is off), a '0' is represented.

To represent characters the bits are combined together. The group of bits representing characters can also be described as **Location**. This location forms the basis of computer storage organization.

Each individual location has got its unique address (identifier). During storage, the address is used to determine the location onto which the character is stored and from which subsequently it is retrieved.

Note. **Reading/retrieving** characters leaves the contents in a location undestroyed, i.e. reading is just copying the contents of a location, leaving the copied contents in their location.

On the other hand, **Writing** to a particular location is destructive in that the previous contents of that location are replaced with the information being written, i.e. The location contents are overwritten, hence cannot be recovered.

The no. of bits that defines the location is influenced by the **computer design** & the **instructions used**.

E.g., most microcomputers were originally designed based on the 8-bit organization. However, due to technological development, modern microcomputers are mostly 16-bit machines.

There are various forms of storage organization.

(i). Character machines & Impacked Binary Fields:

Character machines can also be described as **Character addressable machines**.

In Character machines, each location is made from a group of 6 bits, i.e. the characters are stored in terms of 6 bits. Each character has its own location and hence a unique address for reading or writing individual character.

Character machines use the 6-bit BCD coding system in representing character in the memory.

In Character machines, both characters and numerals are stored in locations of 6 bits. This means that, if the characters are mostly numeric, then a lot of space is wasted because for a numeric character, only 4 bits are necessary to represent the data bits, while the zone bits are just filled in by zeros. This mode of storage location is described as **Impacked binary field** (or **Impacked storage mode**).

(ii). **Word machines.**

The Word machines use fixed word length storage. The typical word lengths are 24 bits or 32 bits depending on the computer design.

The Word machines are designed such that, each word has its own unique location, with a unique address/identifier, which is used to identify the location during writing or reading. One word can store 4 characters.

Word machines have an advantage in the organization of the entities in the computer memory in that, the exact number of characters can be stored and retrieved conveniently, unlike Character machines whose organization of the locations is limited to 6 bits.

(iii). **Byte Machines & Packed Binary Fields:**

A **Byte** is made up of 8 bits and can store one character.

The Byte machines are also referred to as **Byte addressable machines**.

Byte machines use the EBCDIC coding system (the coding system that uses 8 bits to represent characters).

The Byte machines have an advantage over the Character machines in that, if the characters are mostly numeric, they can be stored in packed mode, giving the packed binary fields. In this mode of storage, two characters are stored in single byte, as opposed to impacked fields in Character machines, which stores one character per byte.

For example, 2468 can be packed into 2 bytes (16 bits) as compared to Character machines, which would use 4 bytes (32 bits).

Fig 1: Byte representation of 2468 (packed).

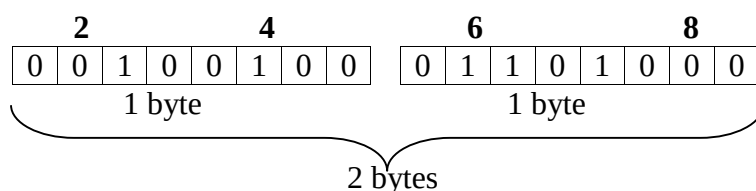
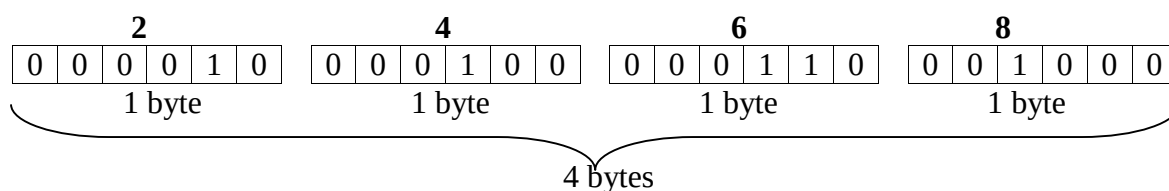


Fig 2: Character machine representation of 2468 (packed).



The arrangement of the locations of a Byte machine provides for a unique identifier for each byte, which can be used for write and retrieval operations on these locations.

The Byte machines can also be described as **Variable Word length machines**. This is because, several bytes can be grouped to form words of required length.

The Byte machines are more advantageous for they combine both the capabilities of the Character and the Word addressable machines.

Exercise I.

1. Expand the following abbreviations:
 - (a). BIT. (1 mark)
 - (b). BCD. (1 mark)
 - (c). ASCII. (1 mark)
 - (d). EBCDIC. (1 mark)
2. Explain the concept of storage organization.
3. Define/Explain the following terms:
 - (i). Word machine.
 - (ii). EPROM.
 - (iii). Byte machine.
 - (iv). Character machine.
 - (v). Packed binary field.
 - (vi). Impacked binary field. (20 marks).
4. Write short notes on:
 - (i). Character machines and impacked binary fields.
 - (ii). Word machines.